# MARI

Motion-Aware Adaptive Range Index

# Paper Progress

Weeks 2–3

Theory, Cost Bounds, Optimality, Coupling,
and Real Competitor Baselines

**Member A(Shah Nadim Kamran Rian):** Theory & Writing
**Member B(Saheeb Tareque):** Implementation & Evaluation

# Contents


**2.1 Objective** **1**

**2.2 Motivation – why this week** **1**

**2.3 Method & key equations** **1**

**2.4 Process (step by step)** **1**

**2.5 Results** **2**

**2.6 Interpretation** **2**

**2.7 Who did what** **2**

**2.8 Deliverables (code)** **2**

**2.9 Status** **2**

**2.10 Transition to Week 3** **2**

**3.1 Objective** **3**

**3.2 Motivation – why this week** **3**

**3.3 Method & key equations** **3**

**3.4 Process (step by step)** **3**

**3.5 Results** **3**

**3.6 Interpretation** **4**

**3.7 Who did what** **4**

**3.8 Deliverables (code)** **4**

**3.9 Status** **4**

**3.10 Next week** **4**

**Combined conclusion** **5**

# MARI – Detailed Weekly Report

Week 2: Theory – Cost Bounds, Optimality & Coupling

Member A (Theory & Writing) • Member B (Implementation & Evaluation)

---

## 2.1. Objective

Prove the mechanism's cost bounds and the result that separates MARI from ordered indexes, and validate them empirically.

## 2.2. Motivation – why this week

A working mechanism is not yet a contribution; a referee asks whether the cost is provably bounded and what fundamentally distinguishes MARI. We supply both, and we report a negative result honestly rather than overclaiming.

## 2.3. Method & key equations

- **Compaction (Theorem 1):**

$$|D_j| \geq \varepsilon |S_j| \quad \Longrightarrow \quad O(1/\varepsilon) \text{ amortized work}, \qquad \text{space } O(n+m).$$

- **Maintenance (Theorem 3):** adaptive split/merge is $O(1)$ amortized.
- **Corollary 4:**

$$\Pr[\text{migrate}] \leq \Pr[|\Delta k| > g].$$

- **Proposition 2 (coupling):** with one width parameter $W$, $r(W) \downarrow$ while $s(W) \uparrow$; MARI decouples guard width $g$ from scan width $w$.

## 2.4. Process (step by step)

1. Prove Theorems 1–3, Corollaries 1–4, Proposition 1 (with its refutation), and Proposition 2 (relocation–scan coupling).
2. Implement adaptive split/merge in `mari_adaptive.py`.
   Run the sanity check with `adaptive_test.py`.
3. Run `thm_check.py` with Python to measure rebuild work per update against the $O(1)$ prediction.
4. Run `lb_check.py` and `lb_frontier.py` to test the tempting, but refuted, lower bound.

## 2.5. Results

| Claim | Statement | Check |
|---|---|---|
| Theorem 1 | Compaction $O(1/\varepsilon)$; space $O(n+m)$ | Matches sweep |
| Theorem 3 | Maintenance $O(1)$ amortized | $\approx 0.27$/update |
| Proposition 2 | Relocation–scan coupling | Separates MARI |

- Theorem 3 was validated at approximately **0.27 rebuild-work per update**, with 0 mismatches.
- Proposition 1's tempting lower bound was *refuted* and retained as an honest negative result.
- Proposition 2 explains why every ordered competitor is pinned to approximately 1.0 relocations per changed update under the tested semantics.

## 2.6. Interpretation

The bounds make MARI predictable rather than heuristic. The coupling result provides the structural explanation: MARI separates the guard width $g$, which controls migration, from the scan width $w$, which controls query granularity. This separation is the central reason that the method can avoid many relocations while preserving exact range reporting.

## 2.7. Who did what

- **Member A:** Proved all theorems and propositions and wrote the theory section, including the honest refutation.
- **Member B:** Built the validation harness and adaptive split/merge implementation and ran the lower-bound checks.

## 2.8. Deliverables (code)

`thm_check.py`, `mari_adaptive.py`, `adaptive_test.py`, `lb_check.py`, and `lb_frontier.py`.

## 2.9. Status

**Status:** Completed.

## 2.10. Transition to Week 3

Week 3 tests the theoretical claims empirically against real ordered-index competitors.

# MARI – Detailed Weekly Report

Week 3: Real Competitor Baselines (ART, PGM, Bx-tree)

Member A (Theory & Writing) • Member B (Implementation & Evaluation)

## 3.1. Objective

Compare MARI head-to-head against faithful, oracle-verified ordered indexes on the same exact-range-under-drift problem.

## 3.2. Motivation – why this week

The strongest reviewer objection is the absence of a competitive baseline. We answer it with real index families: an adaptive radix tree, a dynamic learned index, and a Bx-tree made exact by a verification adapter.

## 3.3. Method & key equations

- **Relocations per update:**

$$\text{MARI } 0.024 \quad \text{versus} \quad 0.990 \quad \Longrightarrow \quad \text{approximately } 40\times \text{ fewer relocations,}$$

  with approximately 110× fewer relocations in the scale experiment.

- **Exactness tax:** ART pays through full relocation; PGM pays through approximately 5× merge writes; and Bx-tree pays through approximately 7× verifications per result.

## 3.4. Process (step by step)

1. Implement ART (Node4/16/48/256), a dynamic PGM (sorted level-0 buffer, verification, and merge garbage collection), and a Bx-tree with an exactness adapter in `competitors.py`.
2. Verify each competitor's answers against the oracle.
3. Run `baseline_bench.py` with Python: five seeds plus a scale run, recording relocation, write, and verification counts.
4. Plot the comparison using `fig_baseline.py`.

## 3.5. Results

| Structure | Reloc./upd. | Writes/upd. | Verify/result | Exact? |
|---|---|---|---|---|
| MARI | 0.024 | 1.15 | 1.01 | Yes |
| ART | 0.990 | 1.08 | 0.00 | Yes |
| PGM | 0.990 | 4.96 | 1.38 | Yes |
| Bx-tree | 0.990 | 1.16 | 6.93 | Yes |

- MARI relocates approximately **40 times less** than every competitor, and approximately **110 times less** in the scale experiment; all four methods remain exact.

- Each competitor preserves exactness by paying elsewhere: ART through relocation, PGM through merge writes, and Bx-tree through additional verification.

## 3.6. Interpretation

The comparison locates MARI's value rather than merely asserting it. The relocation rate is the definitional contrast, while the competitors' different "exactness taxes" demonstrate that exactness is not free. MARI shifts the cost toward guarded ownership, bounded maintenance, and authoritative verification rather than relocating an entry after nearly every non-zero key change.

## 3.7. Who did what

- **Member A:** Defined the shared problem and fairness framing, including implementation-independent metrics and disclosure that the Bx exactness adapter is our construction; wrote the comparison section.
- **Member B:** Implemented all three competitors, verified exactness, ran the five-seed and scale benchmarks, and produced the figure.

## 3.8. Deliverables (code)

`competitors.py`, `baseline_bench.py`, and `fig_baseline.py`.

## 3.9. Status

**Status:** Completed.

## 3.10. Next week

Week 4 checks whether bounded drift is present across independent real-world domains.

## Conclusion

Weeks 2 and 3 jointly move MARI from a working mechanism to a theory-backed and empirically positioned research contribution. Week 2 establishes bounded compaction, space, and maintenance costs and articulates the relocation–scan distinction. Week 3 tests that distinction against three representative ordered-index baselines under a common exactness requirement. Together, the two weeks show that MARI's guarded ownership can substantially reduce structural relocation while retaining exact range-query answers, with the remaining costs made explicit through writes and authoritative verification.